

CloudPoll: Presentación 2

Cloud Computing



UNIVERSIDAD
CATÓLICA
BOLIVIANA

ERIGIDA CANÓNICAMENTE
POR LA SANTA SEDE DESDE 2023

Integrantes: Rodrigo Gabriel Rivera Mayan

Jose Andres Manzaneda Manzaneda

Leandro Baldur Colque Palacios

Fecha de entrega: 19 de Mayo de 2026

Docente: Juan Jose Santos Miranda Mendoza

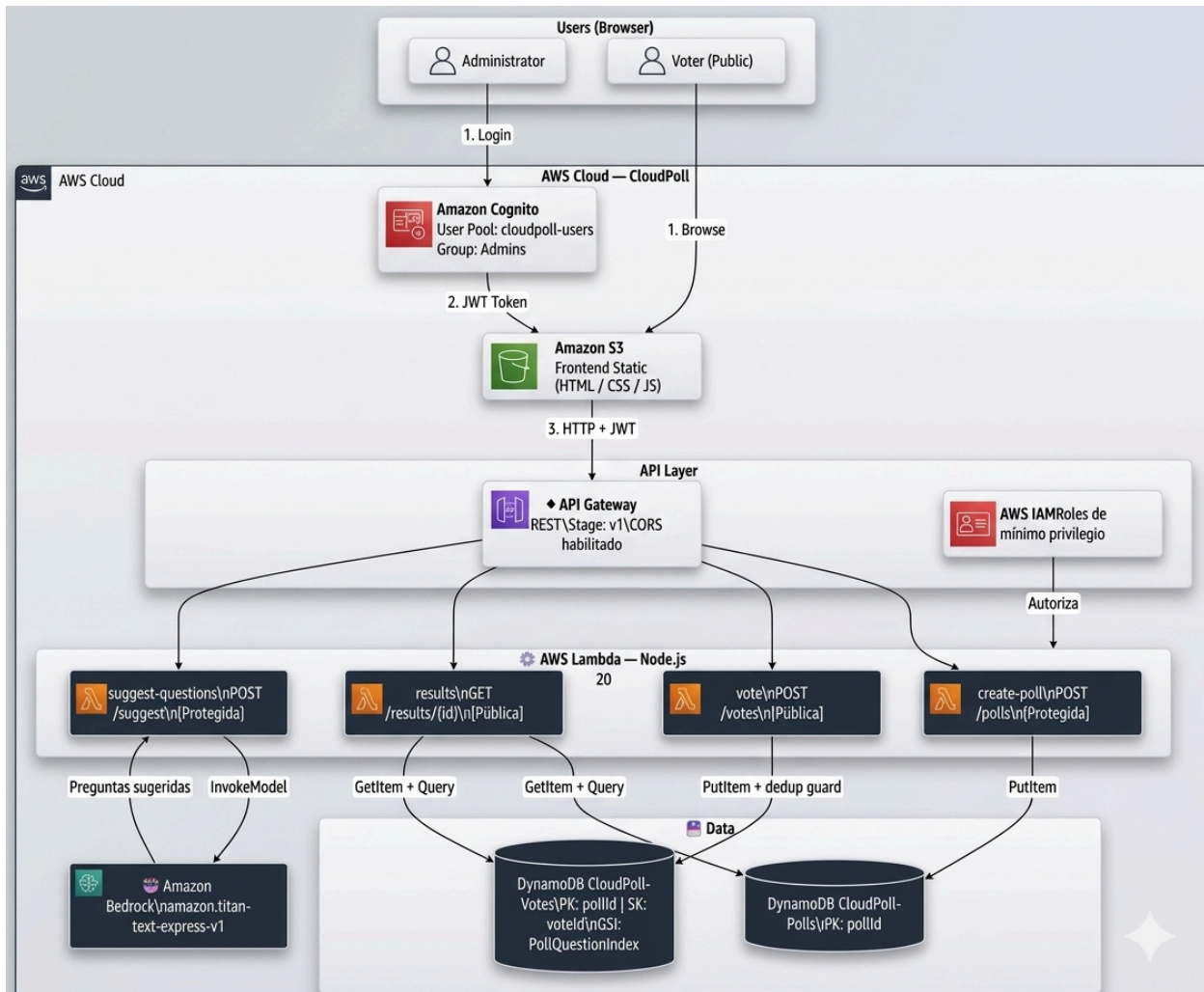
CloudPoll — Presentación 2: Arquitectura Detallada y Avance Real

1. Arquitectura Detallada

1.1 Diagrama de flujo de datos

Arquitectura general del sistema

- **Usuarios:**
 - Administrador.
 - Votante público.
- **AWS Cloud — CloudPoll:**
 - **Amazon Cognito:**
 - *User Pool*: cloudpoll-users
 - Grupo: Admins
 - **Amazon S3:**
 - Alojamiento (hosting) estático del *frontend* (HTML / CSS / JavaScript).
- **Capa de API (API Layer):**
 - **API Gateway REST:**
 - Stage: v1
 - CORS habilitado.
 - **AWS IAM:**
 - Roles de mínimo privilegio.
- **Capa de Cómputo (AWS Lambda — Node.js 20):**
 - create-poll: POST /polls (Ruta protegida).
 - vote: POST /votes (Ruta pública).
 - results: GET /results/{id} (Ruta pública).
 - suggest-questions: POST /suggest (Ruta protegida).
- **Persistencia de datos:**
 - **DynamoDB CloudPoll-Polls**: PK: pollId
 - **DynamoDB CloudPoll-Votes**: PK: pollId, SK: voteId, GSI: PollQuestionIndex
- **Inteligencia Artificial:**
 - **Amazon Bedrock**: Modelo amazon.titan-text-express-v1



Descripción: Diagrama de Flujo de Datos

Flujo principal:

1. El administrador inicia sesión mediante Amazon Cognito.
2. Cognito devuelve un token JWT al *frontend*.
3. El *frontend* hospedado en S3 consume los servicios a través de API Gateway.
4. API Gateway valida el JWT usando Cognito.
5. AWS Lambda procesa las solicitudes entrantes.
6. DynamoDB almacena la información de las encuestas y los votos emitidos.
7. Amazon Bedrock genera sugerencias de preguntas mediante Inteligencia Artificial.

1.2 Flujo de datos por caso de uso

Caso 1 — Votante emite un voto

- **Acción del navegador:** Petición POST /v1/votes
- **Cuerpo de la petición (Payload):**

```
{
  "pollId": "pollId",
  "questionId": "questionId",

```

```
"optionId": "optionId"
}
```

- **Proceso:**

1. API Gateway recibe la petición pública.
2. La función Lambda vote verifica si existe la clave de deduplicación: DUP#<ip>#<pollId>.
3. **Si el usuario ya votó:** Retorna código HTTP 409 Conflict (Ya votaste).
4. **Si no existe duplicado:** Se registra el voto en DynamoDB y se crea el ítem de protección de deduplicación.

- **Respuesta Exitosa:**

```
{
  "voteId": "...",
  "timestamp": "..."
}
```

Caso 2 — Admin crea una encuesta

- **Acción del navegador:** Petición POST /v1/polls
- **Cabecera (Header):** Authorization: Bearer <JWT>
- **Cuerpo de la petición:**

```
{
  "title": "Encuesta",
  "description": "Descripción",
  "questions": []
}
```

- **Proceso:**

1. API Gateway valida el token JWT con Cognito.
2. **Si el JWT es inválido:** Retorna código HTTP 401 Unauthorized.
3. **Si es válido:** La función Lambda create-poll valida los datos, genera un UUID y guarda la encuesta en DynamoDB.

- **Respuesta Exitosa:**

```
{
  "pollId": "...",
  "createdAt": "..."
}
```

Caso 3 — Consulta de resultados

- **Acción del navegador:** Petición GET /v1/results/{pollId}
- **Proceso:**
 1. API Gateway envía la solicitud a la función Lambda results.
 2. Lambda obtiene los metadatos de la encuesta desde CloudPoll-Polls.
 3. Consulta los votos emitidos desde CloudPoll-Votes.
 4. Calcula los conteos y porcentajes de cada opción.
- **Respuesta Exitosa:**

```
{
```

```
"title": "...",
"results": []
}
```

1.3 Integraciones entre servicios

Origen	Destino	Protocolo	Datos transmitidos
S3 (<i>Frontend</i>)	Cognito	HTTPS Redirect	Código OAuth
Cognito	<i>Frontend</i>	HTTPS	JWT (JSON Web Token)
<i>Frontend</i>	API Gateway	HTTPS REST	JSON + Bearer Token
API Gateway	Cognito	Interno AWS	Validación JWT
API Gateway	Lambda	Invocación	Evento JSON
Lambda	DynamoDB	AWS SDK v3	PutItem / GetItem / Query
Lambda	Bedrock	AWS SDK v3	Prompt → JSON
Lambda	IAM	Implicito	Assume Role

2. Decisiones Técnicas

2.1 Serverless vs. Arquitectura Tradicional

Criterio	Serverless (CloudPoll)	Tradicional (EC2 + RDS)
Escalabilidad	Automática	Manual / Auto Scaling
Disponibilidad	Alta disponibilidad (nativa de AWS)	Depende de la configuración
Costo en reposo	\$0	~\$15–50/mes
Costo en pico	Según uso (Pago por ejecución)	Siempre fijo (infraestructura encendida)
Despliegue	Automatizado (sam deploy)	Configuración manual /

		Pipelines complejos
Mantenimiento	Administrado por AWS	Administrado por el equipo de desarrollo
Arranque en frío	100–500 ms (<i>Cold Start</i>)	No existe
Tiempo de ejecución	Máximo 15 minutos (límite Lambda)	Sin límite
Estado	<i>Stateless</i> (Sin estado)	Puede mantener estado en memoria

¿Por qué Serverless para CloudPoll?

1. **Tráfico impredecible:** Una encuesta viral puede pasar de 0 a miles de votos en minutos. Lambda escala automáticamente sin intervención humana.
2. **MVP económico:** La capa gratuita de AWS permite validar el producto prácticamente sin costos operativos iniciales.
3. **Equipo pequeño:** No se requiere administrar servidores, parches del sistema operativo ni infraestructura subyacente.
4. **Sinergia con DynamoDB:** El modelo NoSQL encaja naturalmente en aplicaciones escalables y sin servidores.
5. **Simplicidad con Cognito:** Manejo de JWT, MFA y usuarios integrados sin necesidad de desarrollar módulos de autenticación desde cero.

Trade-offs (Compromisos) aceptados

- Presencia de *Cold Starts* (Arranques en frío) en las funciones Lambda.
- Alta dependencia tecnológica (*Vendor Lock-in*) con AWS.
- Obtener resultados en tiempo real requerirá técnicas de *polling* (peticiones periódicas) o la futura implementación de WebSockets.

3. Avance Funcional

3.1 Servicios implementados

#	Servicio AWS	Estado
1	DynamoDB Local	Tablas creadas
2	Lambda results	Calcula porcentajes
3	Lambda vote	Bloquea duplicados
4	Lambda create-poll	Genera UUID
5	Lambda suggest-questions	Pendiente integración con

		Bedrock
6	API Gateway local	Endpoints activos
7	Repositorio GitHub	Publicado

3.2 Estructura del repositorio

```

CloudPoll/
├── template.yml
├── docker-compose.yml
├── env.local.json
├── scripts/
│   ├── setup-local.js
│   └── seed-local.js
├── lambdas/
│   ├── create-poll/index.js
│   ├── vote/index.js
│   ├── results/index.js
│   └── suggest-questions/index.js
└── docs/
    ├── architecture.md
    ├── TechStack.md
    ├── Components.md
    ├── Requisitos.md
    └── implementation-guide.md

```

3.3 Endpoints disponibles

Público — Registrar voto

- POST `http://localhost:3000/votes`

```

{
  "pollId": "poll-test-001",
  "questionId": "q1",
  "optionId": "o1"
}

```

Respuesta: `{"message": "Voto registrado", "voteId": "...", "timestamp": "..."}"`

Público — Ver resultados

- GET `http://localhost:3000/results/poll-test-001`

Protegido — Crear encuesta

- POST `http://localhost:3000/polls`

- *Header:* Authorization: Bearer <token>

Protegido — IA para preguntas

- POST http://localhost:3000/suggest
- *Header:* Authorization: Bearer <token>

```
{
  "topic": "tecnología web",
  "numQuestions": 3
}
```

3.4 Datos de prueba insertados

Tabla CloudPoll-Polls

```
{
  "pollId": "poll-test-001",
  "title": "Encuesta de tecnologías favoritas",
  "status": "active"
}
```

Tabla CloudPoll-Votes

voteld	questionId	optionId	voterIp
v001	q1	o1	10.0.0.1
v002	q1	o1	10.0.0.2
v003	q1	o2	10.0.0.3
v004	q1	o3	10.0.0.4

3.5 Logs de ejecución local (SAM)

Mounting ResultsFunction at

http://127.0.0.1:3000/results/{pollId}

Mounting VoteFunction at http://127.0.0.1:3000/votes

Mounting CreatePollFunction at http://127.0.0.1:3000/polls

Mounting SuggestQuestionsFunction at

http://127.0.0.1:3000/suggest

Running on http://127.0.0.1:3000

4. Repositorio con Código

- **Repositorio GitHub:** [CloudPoll]
- **Commits relevantes:**
 - docs: arquitectura, requisitos, componentes
 - feat: template SAM completo
 - feat: implementación Lambdas Node.js
 - feat: DynamoDB Local + Docker

5. Estimación de Costos

Escenario base mensual:

- 10,000 encuestas creadas.
- 100,000 votos emitidos.
- 500 consultas de resultados por día (~15,000/mes).

Detalle por Servicio

Servicio	Parámetro	Valor Estimado	Costo Estimado
AWS Lambda	Invocaciones	~220,000	\$0.00
	Duración media	300 ms	
	Memoria	256 MB	
DynamoDB	Escrituras	~110,000	\$0.00
	Lecturas	~215,000	
	Almacenamiento	< 1 GB	
API Gateway	Llamadas a la API	~220,000	\$0.00
Cognito	Usuarios activos (MAU)	~10 (Solo administradores)	\$0.00
Amazon S3	Peso del Frontend	< 10 MB	\$0.00
	Peticiones GET	~15,000	
Amazon Bedrock	Uso del modelo <i>Titan Text Express</i>	~200 consultas/mes	~\$0.04/mes

5.7 Resumen de costos

Servicio	Escenario MVP	Escala 10x (1M votos/mes)
Lambda	\$0.00	~\$1.20
DynamoDB	\$0.00	~\$2.50
API Gateway	\$0.00	~\$0.70
Cognito	\$0.00	\$0.00
S3	\$0.00	~\$0.05
Bedrock	~\$0.04	~\$0.40
TOTAL ESTIMADO	~\$0.04	~\$4.85

6. Coherencia del Sistema

¿Ya funciona algo real?

- **Sí.** DynamoDB Local está operativo mediante Docker.
- Las funciones Lambda principales están codificadas y operativas.
- API Gateway local enruta correctamente las peticiones.
- La lógica de deduplicación de votos está validada.
- Los resultados calculan y emiten los porcentajes en tiempo real.

¿Las decisiones están justificadas?

- **Costo:** La arquitectura *Serverless* garantiza costos casi nulos durante la fase de validación (MVP).
- **Concurrencia:** DynamoDB fue elegido específicamente por su capacidad para manejar picos masivos de escrituras (votos).
- **Seguridad:** Cognito abstrae el esfuerzo de desarrollar autenticación personalizada, mientras que API Gateway centraliza la validación de tokens.

¿El sistema es coherente?

- El diseño respeta el principio de responsabilidad única (cada Lambda hace una sola cosa).
- Se aplica el principio de *Mínimo Privilegio* a través de IAM.
- Existe una separación clara entre las rutas públicas (votación/resultados) y protegidas (creación/sugerencias de IA).
- El *frontend* está completamente desacoplado del *backend*, operando a través de contratos de API claros (JSON).